

(Randomized) Quicksort

Design and Analysis of Algorithms

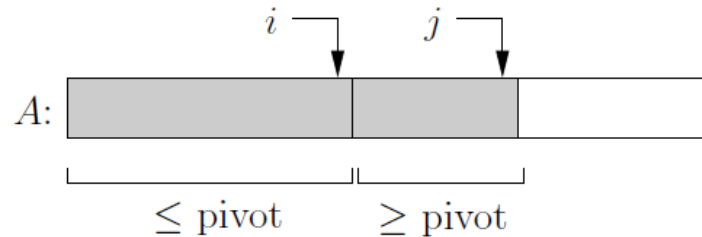
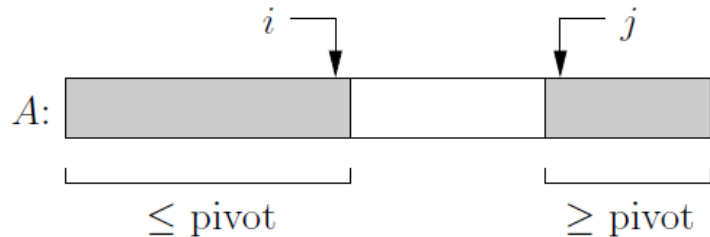
Brian C. Dean

**School of Computing
Clemson University**



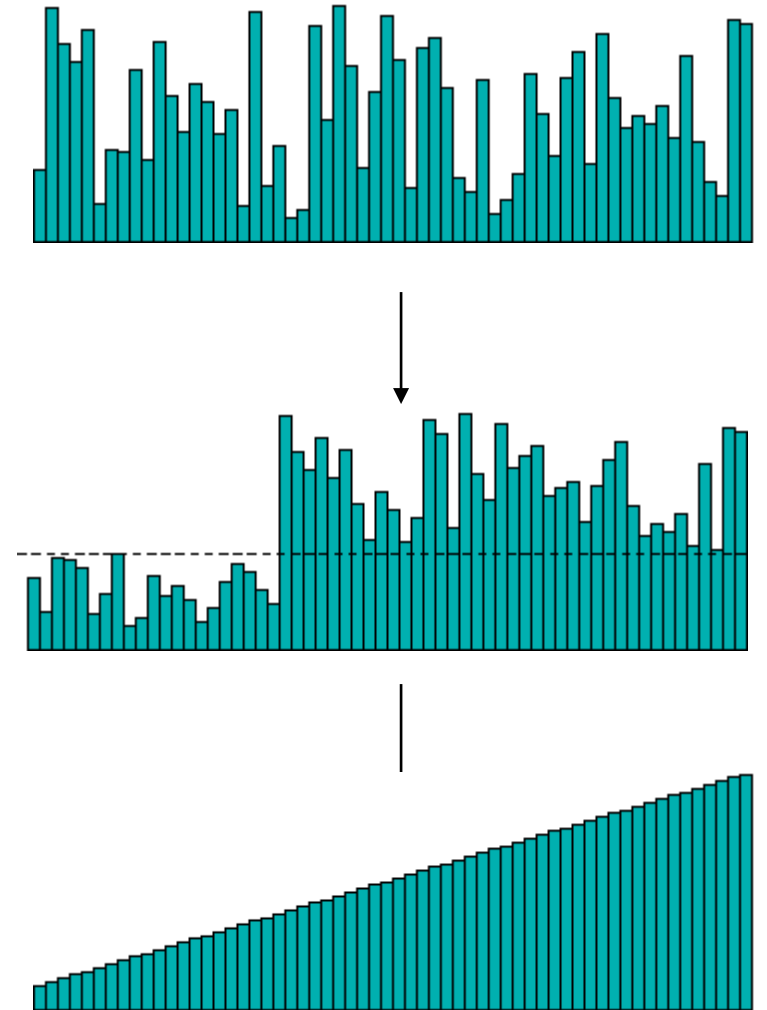
Quicksort (Divide and Conquer)

- **Partition** array based on the value of some **pivot** element.
- Then recursively sort left side ($\leq$ pivot) and right side ($\geq$ pivot).
- Easy to partition in $O(n)$ time.



- If we partition “in place”, this makes the entire algorithm run “in place” (*)

(*) Ignoring stack space due to recursion



Running Time

- Suppose every partition is “bad” (1 vs. $n-1$)
 - $\Theta(n^2)$ time (this is the worst case for quicksort)
- Suppose every partition “good” (constant fraction reduction on both sides, like $\frac{1}{4} : \frac{3}{4}$)
 - Each element takes part in $\Theta(\log n)$ partitions.
 - Each element undergoes $O(1)$ work per partition.
 - Hence, $O(n \log n)$ total time.
- It is possible to partition on the median ($\frac{1}{2} : \frac{1}{2}$ split), leading to $\Theta(n \log n)$ worst-case running time, but this isn't often done since it's complicated and has a high hidden constant.

Quicksort Variants Based on Pivot Choice

- **Pivot = median of array**
 - $\Theta(n \log n)$ time, but complicated and high hidden constant.
- **Pivot = median($A[1]$, $A[n]$, $A[n/2]$)**
 - Reasonable heuristic. Usually fast in practice, although still $\Theta(n^2)$ worst-case against a truly adversarial input.
- **Pivot = random element.**
 - $\Theta(n \log n)$ time both in expectation and “with high probability”
 - Each element still undergoes $O(1)$ work per partition, and its subproblem size shrinks according to randomized reduction, so $O(\log n)$ expected partitions $\rightarrow O(n \log n)$ expected total time.

Randomized Reduction and Randomized Quicksort

Randomized binary search for 7

1	2	3	7	8	9	12	13	14	15
---	---	---	---	---	---	----	----	----	----

1	2	3	7	8	9
---	---	---	---	---	---

3	7	8	9
---	---	---	---

3	7
---	---

Remember we showed this involved $O(\log n)$ steps by the randomized reduction lemma...

--

 = random choices of “pivots”

Randomized quicksort

3	7	9	2	8	1	12	14	15	13
---	---	---	---	---	---	----	----	----	----

1	2	8	7	9	3
---	---	---	---	---	---

13	14	15
----	----	----

1	7	3	8	9
---	---	---	---	---

13

15

3	7
---	---

9

3

--

 = random choices of pivots